

DSA STUDY BLUEPRINT SERIES

111. Detect Cycle in Directed Graph using DFS

Checking Cycles in Directed Graphs using DFS Tracker

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Cycle checks in directed graphs.
- **DFS Path Tracker:** Maintain a visited array and a recursion path tracker array.

Memory & Execution Blueprint

Path Visited Set tracks active recursion path: [1 → 2 → 3 → 1 (Cycle detected!)]

C++ Implementation Reference

Standard code layout and syntax

```
bool hasCycleDir(int u, vector<int>& adj, vector<bool>& vis, vector<bool>& pathVis) {  
    vis[u] = pathVis[u] = true;  
    for (int v : adj[u]) {  
        if (!vis[v]) {  
            if (hasCycleDir(v, adj, vis, pathVis)) return true;  
        } else if (pathVis[v]) return true; // Cycle found  
    }  
    pathVis[u] = false; // backtrack  
    return false;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

Key Practices & Gotchas

- **Important:** Reset path tracker values (`pathVis[u] = false`) after recursive path returns.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.